Review 1

Agentic Shell: Agent 10 – Inode & Symlink Auditor Agent

A natural-language agent for finding broken links, symlink loops and hard links on Linux

Name	Registration No.
Debojyoti Barai Mitul	24BCE2951
Shivam Prakash	24BDS0340

Chapter 1: Introduction

1.1 Project Overview

This project is **Agent 10, the Inode & Symlink Auditor Agent**, one of the 35 agents of our class project, the *Agentic Shell*. In the Agentic Shell, a base shell receives a request written in plain English and routes it to the agent best suited to handle it. Our agent is responsible for everything to do with inodes and links on a Linux system: it finds broken ("dangling") symbolic links and symlink loops, groups files that are hard links of each other, inspects the inode, link count, owner and permissions of any path, reports how many inodes a filesystem has left, flags suspicious links, and, only after the user confirms, deletes dead links or creates and repoints links. A typical request is *"Find all broken shortcuts on my desktop and delete them."*

Technically, the agent exposes Linux capabilities as **tools for LLM function calling**. A large language model (LLM) reads the request, decides which tools to run and with which arguments, and our Python code performs the real work with system calls such as `lstat()`, `readlink()`, `statvfs()` and `unlink()`. The agent runs on its own today (a chat mode, one-shot requests and direct commands that need no LLM) and offers a small Python interface so that whatever base shell the class adopts can plug it in.

The evolution of Command Line Interfaces (CLIs)

Command line interfaces are the oldest interactive way of controlling a computer. Early time-sharing systems were driven from teletype terminals, where the user typed a command and the machine printed a reply. Unix turned this into a programmable environment: the Thompson shell of the early 1970s was followed by the Bourne shell (1979), which added scripting constructs, and later by the C shell, the Korn shell and GNU **Bash** (1989), which is still the default interactive shell on most Linux distributions. Microsoft followed a different path with **PowerShell** (2006), which passes structured objects between commands instead of plain text.

Across these generations the CLI kept the same strengths: it is fast, scriptable, precise and works over a remote connection. It also kept the same weakness: the user must already know the exact command, its options and its quoting rules. Modern terminals improved the experience with tab completion, history search and syntax highlighting, but they still expect the user to speak the shell's language.

The shift toward Natural Language Processing (NLP) in system administration

Natural language interfaces try to reverse that relationship, so that the computer understands the user's language instead. Early research such as the **NL2Bash** dataset (Lin et al., 2018) showed

that translating English into correct Bash commands is hard, because a single request can map to many commands and a small mistake changes the meaning. Large language models trained on code changed what is practical: since around 2021, models such as OpenAI Codex (used by GitHub Copilot) can write working shell commands from a description, and since 2023 most LLM APIs support **function calling** (also called tool calling), where the model returns a structured request to call a named function with JSON arguments instead of free text.

Function calling is the basis of today's *agents*: programs in which an LLM plans the steps and calls tools, while ordinary code carries out the actions. In system administration this means a user can ask *"Am I running out of inodes?"* and the agent decides to measure inode usage, reads the result and answers in plain words. Our Agentic Shell applies this idea to a whole shell, and our agent applies it to one specific, error-prone area: links and inodes.

1.2 Problem Statement

Syntactical barriers and steep learning curves of traditional shells (Bash/PowerShell)

Inode and link maintenance is a good example of tasks that are simple to describe but hard to type. Each question needs a different tool and a different set of options:

What the user wants	Typical shell command
Find broken symbolic links in a folder	`find ~/Desktop -xtype l`
Find every name of the same file (hard links)	`ls -i file`, then `find / -xdev -inum N` or `find / -samefile file`
See how many inodes are left	`df -i /home`
See where a link really points	`readlink -f link`, `stat link`
Find links that loop forever	no single standard command; usually a script around `find -L` and error messages

Table 1.1: Everyday link and inode questions and the commands they need

A beginner has to know that `-xtype l` (not `-type l`) finds broken links, that `-xdev` keeps `find` on one filesystem, and that inode numbers are only unique within a filesystem. PowerShell users face a different but equally steep syntax. As a result, many users never check for dangling shortcuts, duplicated hard links or inode exhaustion until something fails, for example when a disk with free space reports *"No space left on device"* because it has run out of inodes.

Risk of executing incorrect commands due to human error

The commands that clean up links are also the commands that can destroy data. Many tools behave differently depending on whether they follow symbolic links: `find -L` follows them and can wander into, or loop through, other directories; a trailing slash on a link to a directory (`link/`) makes many commands act on the target directory instead of the link itself; and a wrongly quoted wildcard in an `rm` command can delete far more than intended. Hard links add a further trap: deleting one name of a hard-linked file does not free its data, while editing it changes every other name at once.

Letting an LLM generate raw shell commands does not remove these risks; it adds new ones. A model can invent options, guess a path the user never mentioned, or produce a command that is correct in one shell and harmful in another. A file name controlled by someone else can even

contain text that tries to instruct the model (prompt injection). The problem this project addresses is therefore: **how can a user manage links and inodes in plain English without having to learn the commands, while making it impossible for either the user or the LLM to delete or damage the wrong thing?**

1.3 Objectives

To build a lightweight, secure NL-to-CLI translation engine

Our engine translates natural language into **typed tool calls rather than shell text**. The LLM never writes a command line; it can only choose one of eleven well-defined functions and fill in its arguments, which our code validates. This keeps the engine secure, because nothing the model writes is executed as a command, and lightweight, because it is written in Python 3.8+ using only the standard library: no `pip install`, no SDK and no `sudo`, so it runs on a fresh Ubuntu lab PC.

To integrate context-aware tools using the Model Context Protocol (MCP)

Each tool is described by a name, a plain-English description and a JSON Schema for its arguments, which is the same contract the Model Context Protocol uses for MCP tools (Section 2.2). In the current version the agent offers these tools to the LLM through the OpenAI-compatible function-calling API directly, because the class LLM was unknown and the project had to use the standard library only; the tool layer is kept independent of the LLM layer so that it can be published as an MCP server later without changing the tools. The tools are **context-aware**: the agent keeps a short session note with the current folder, the last actions and the items found, so follow-ups such as *"delete them"* or *"what about that folder?"* are understood.

Specific objectives

1. Detect dangling symlinks and symlink loops (including self-links and cycles such as A → B → A) with `lstat()`, `readlink()` and the `ELOOP` error, without ever following links while scanning.
2. Group hard links by (device, inode), and report inode details and filesystem inode usage (like `df -i`) through `os.statvfs()`.
3. Flag suspicious links: links that leave a chosen folder, point into sensitive places such as `/etc` or `~/.ssh`, or live in world-writable directories.
4. Make every change safe: preview first, ask the user y/N, refuse protected system paths and files the user does not own, re-check right before acting, and log every action.
5. Work standalone (chat mode, one-shot requests and direct commands without an LLM) and be easy to plug into any base shell through a small Python API.
6. Work with any OpenAI-compatible LLM provider (for example Groq, OpenAI or a local Ollama model) and handle rate limits gracefully.

1.4 Scope and Limitations

Supported operating systems and terminal environments

- **Operating system:** Linux. The agent was developed and tested on Ubuntu with Python 3.8 or newer; it relies on POSIX behaviour of symbolic links, hard links, inodes and `statvfs()`.

- **Terminals:** any terminal emulator and shell (for example GNOME Terminal with Bash or Zsh). Chat mode uses Python's `input()` with `readline` for line editing; output is plain text so it reads well in any terminal.
- **Non-interactive use:** the agent can be called by another program (the base shell). In that case it never writes anything unless the caller supplies a confirmation callback or explicitly passes `auto_confirm=True`.
- **Not covered:** Windows and macOS were not targeted or tested. Windows handles symbolic links differently (they may require special privileges), and macOS was not available for testing.

Extent of administrative privileges granted to the AI model

The AI model is given **no shell access and no administrative privileges**. It can only request one of eleven tools: seven are read-only, and the four that change anything (delete dead links, create a symbolic link, create a hard link, repoint a link) all pass through the same safety gate. The program runs as the normal user, never uses `sudo`, refuses to write inside `/`, `/bin`, `/boot`, `/dev`, `/etc`, `/lib*`, `/proc`, `/sbin`, `/sys`, `/usr` and `/var`, refuses anything the current user does not own, and only deletes paths that are still symbolic links that still fail to resolve at the moment of deletion. The model cannot confirm a change itself: the y/N question is asked by our program.

Limitations

- Hard links can only be grouped among the names inside the scanned folder; other names elsewhere on the filesystem need a wider scan (the report states how many names are outside).
- Scans are bounded by a depth limit, an entry limit and a time limit so that scanning `/` or the home folder cannot hang; very large trees may therefore be reported as truncated.
- Natural-language mode needs network access and an API key. Free tiers impose rate limits (for example 8,000 tokens per minute on Groq's free tier), so the agent keeps requests small and retries automatically, but very fast conversations may still have to wait. Direct mode works without any LLM.
- The agent manages links and inodes only; it does not delete regular files or directories, change permissions or move files.

Chapter 2: Literature Review & Theoretical Background

2.1 Large Language Models (LLMs) in Code Generation

Capabilities of modern LLMs in structured text and code synthesis

Large language models are transformer neural networks trained on very large collections of text, including source code. Models trained or fine-tuned on code, starting with OpenAI Codex (2021), showed that an LLM can turn a short description into a working function, a SQL query or a shell command, and today's general-purpose models (for example GPT-4-class models, Claude, Llama and Gemini) do this routinely. Two capabilities matter most for an agent like ours:

- **Structured output.** Models can produce output that follows a given format, such as JSON that matches a schema, which a program can parse reliably.

- **Function (tool) calling.** The application sends the model a list of tools, each with a name, a description and a JSON Schema for its parameters. Instead of answering in prose, the model can reply with a request such as `{"name": "find_broken_symlinks", "arguments": {"path": "~/Desktop"}}`. The application runs the function and sends the result back, and the model continues until it can answer the user. This pattern is supported by the OpenAI Chat Completions API and by many compatible providers, including Groq and Ollama.

Because the model only chooses *what* to do and the application decides *how* it is done, function calling lets an LLM operate a real system through a narrow, well-defined interface.

Challenges with LLM hallucination in system-level execution

LLMs can *hallucinate*: they produce fluent, confident output that is wrong. In system administration this is dangerous because the output may be executed. Typical failures include inventing command options that do not exist, guessing a path the user never mentioned, choosing a broader action than requested (deleting a folder instead of a link), misreading earlier results, and following instructions hidden in data, for example a file named so that it reads like a command to the model (prompt injection). Research on natural-language-to-Bash translation such as NL2Bash also showed that even correct-looking commands often differ in small but important details.

For this reason our design treats the LLM as an untrusted planner. The model can only call fixed tools with validated arguments; it is told never to guess paths; every write is previewed and confirmed by the user through our program; and every deletion is re-checked by code immediately before it happens. Tool results are passed to the model as data, never as instructions.

2.2 Introduction to Model Context Protocol (MCP)

Architecture of MCP (Clients, Servers, and Tools)

The **Model Context Protocol** is an open standard, introduced by Anthropic in November 2024, for connecting AI applications to external tools and data. It defines three roles:

- **Host:** the AI application the user works with (for example a desktop assistant, an IDE or a shell). The host contains the LLM integration and decides what the model may use.
- **Client:** a component inside the host that maintains a connection to exactly one server and speaks the protocol on the host's behalf.
- **Server:** a program that exposes capabilities. The main capability types are **tools** (functions the model can call, each described by a name, a description and a JSON Schema for its input), **resources** (data the application can read, such as files) and **prompts** (reusable prompt templates).

Messages are exchanged with **JSON-RPC 2.0**. A client can ask a server which tools it offers (`tools/list`) and call one (`tools/call`). Local servers usually communicate over standard input and output (the *stdio* transport) as a child process, while remote servers use HTTP. Because tools are discovered at run time, a host can use any MCP server without code written specifically for it.

Why MCP is preferred over traditional rigid API tooling for local system access

- **One standard instead of many integrations.** With traditional tooling every application needs custom glue code for every API. An MCP server written once can be used by any MCP-capable host.
- **Self-describing tools.** Names, descriptions and JSON Schemas are discovered at run time, so the model always sees the current, exact interface.
- **Local and contained.** A local server runs as a separate process over stdio, so the host gets access to local system functions without opening a network port, and the server itself decides what is exposed.
- **Human oversight.** The host remains in control of which servers and tools are enabled and can ask the user before a tool runs.

Relation to our project. Our eleven tools follow exactly this tool model: each is a plain function with a name, a description and a JSON Schema, and each returns a JSON-serialisable result. Because the class LLM was unknown and the project was limited to the Python standard library, the agent currently offers the tools through the OpenAI-compatible function-calling API, which needs no SDK. The tool layer (`tools.py` and `schemas.py`) is separate from the LLM layer, so exposing the same tools through an MCP server is a straightforward next step.

2.3 Existing Solutions

Comparative study of tools like GitHub Copilot CLI, Warp AI, and open-source alternatives

Several tools already bring LLMs into the terminal. They are general-purpose: the model writes a command or a program, and the user is asked before it runs.

- **GitHub Copilot in the CLI.** GitHub's command-line assistant suggests and explains shell, Git and GitHub CLI commands from a natural-language description (originally the `gh copilot suggest` and `gh copilot explain` commands, with newer agent-style versions that can also run commands after asking). It requires a GitHub Copilot subscription and a network connection.
- **Warp AI.** Warp is a modern terminal application with built-in AI features: natural-language command search and an agent mode that can propose and run multi-step command sequences, with the user able to review them first. It is a complete terminal rather than a component that can be added to another shell.
- **ShellGPT (open source).** A Python command-line tool that sends a request to an LLM and, in shell mode, prints a suggested command and asks whether to execute it, describe it or abort.
- **Open Interpreter (open source).** Lets an LLM write and run code (Python, shell and others) on the local machine, asking for confirmation before running code unless auto-run is enabled.

Tool	How the LLM acts	Safety approach	Fit for link / inode auditing
GitHub Copilot CLI	Generates shell commands	User reviews and approves the command	General; user must judge the command
Warp AI	Generates and runs	Review before running	General; tied to the Warp

Tool	How the LLM acts	Safety approach	Fit for link / inode auditing
	command sequences		terminal
ShellGPT	Generates one shell command	Execute / describe / abort prompt	General; command text is not checked
Open Interpreter	Writes and runs code	Confirmation before running code	Very general; broad access
Our agent (Agent 10)	Calls 11 fixed, typed tools	Preview, y/N by the program, protected paths, ownership check, re-check before acting, log	Specialised: broken links, loops, hard links, inodes

Table 2.1: Comparison of existing LLM terminal assistants with our agent

Gap addressed by this project. Existing assistants ask the user to approve *generated command text*, so safety depends on the user spotting a mistake in a command they may not understand. Our agent never produces command text. It offers a small set of audited tools whose behaviour is fixed in code, shows a plain-language preview of exactly which links will change and why ("target missing" or "symlink loop"), and re-checks each path at the moment of action. It is also designed as a component: any base shell can call it through `handle()` or `call_tool()`.

2.4 Theoretical Background: Inodes, Hard Links and Symbolic Links

An **inode** is the data structure a Linux filesystem uses to describe a file: its type, size, owner, permissions, timestamps, link count and where its data is stored. A file name is only a directory entry that points to an inode, and inode numbers are unique only within one filesystem, so a file is identified by the pair (device, inode).

- A **hard link** is an additional directory entry for the same inode. All names are equal; the data is freed only when the link count drops to zero. Hard links cannot cross filesystems and cannot normally point to directories.
- A **symbolic (soft) link** is a separate small file whose content is a path. If the target is moved or deleted, the link becomes **dangling (broken)**. If links point at each other (A → B → A, or a link to itself), resolving them fails with the `ELOOP` error; Linux gives up after 40 links in a chain.
- `stat()` follows symbolic links, while `lstat()` describes the link itself and `readlink()` returns its stored path. An auditor must use `lstat()` while walking directories, otherwise it may follow a link into another part of the system or loop forever.
- Every filesystem has a fixed or limited number of inodes. `statvfs()` (used by `df -i`) reports total and free inodes; if they run out, no new files can be created even when disk space is free.

Property	Hard link	Symbolic link
Has its own inode	No (shares the target's inode)	Yes
Survives deletion of the "original" name	Yes	No, it becomes dangling

Property	Hard link	Symbolic link
Can cross filesystems	No	Yes
Can point to a directory	No (normally)	Yes
How our agent detects it	Same (device, inode), link count > 1	`lstat()` + `readlink()`; broken if the target is missing, loop if `ELOOP`

Table 2.2: Hard links compared with symbolic links

These system calls are exactly what the agent's tools use, which is why the project can work with only the Python standard library modules `os`, `os.path` and `stat`.

References

- X. V. Lin, C. Wang, L. Zettlemoyer and M. D. Ernst, "NL2Bash: A Corpus and Semantic Parser for Natural Language Interface to the Linux Operating System," *Proc. LREC*, 2018.
- M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021 (OpenAI Codex).
- Anthropic, "Introducing the Model Context Protocol," November 2024; Model Context Protocol specification, modelcontextprotocol.io.
- OpenAI, "Function calling" (Chat Completions API documentation).
- Linux manual pages: `inode(7)`, `symlink(7)`, `path_resolution(7)`, `stat(2)`, `statvfs(3)`, `find(1)`, `df(1)`.
- Project repository: github.com/Debojyoti17/inode-symlink-auditor-agent (private). The code was developed with AI assistance (Claude), which the course permits.